Initialisation et propriétés avancées en Swift

420-249-AH - Programmation mobile iOS

1. Rappel : pourquoi utiliser init ?

Une classe peut avoir besoin de certaines valeurs dès sa création.

Exemple :

```
class Utilisateur {
    var nom: String
    var age: Int

    init(nom: String, age: Int) {
        self.nom = nom
        self.age = age
    }
}
```

Création :

```
let utilisateur = Utilisateur(
    nom: "Alex",
    age: 25
)
```

L'initialisateur permet de créer l'objet directement dans un état cohérent.

2. Initialisation obligatoire des propriétés

En Swift, les propriétés stockées non optionnelles doivent avoir une valeur avant la fin de l'initialisation.

Ceci est incorrect :

```
class Appareil {
    var nom: String
    var batterie: Int
}
```

Pourquoi ? Parce que Swift ne sait pas quelles valeurs donner à nom et batterie.

3. Deux solutions principales

Donner une valeur par défaut

```
class Appareil {
    var nom: String = "Inconnu"
    var batterie: Int = 100
}
```

Création :

```
let appareil = Appareil()
```

Utiliser un initialisateur

```
class Appareil {
    var nom: String
    var batterie: Int

    init(nom: String, batterie: Int) {
        self.nom = nom
        self.batterie = batterie
    }
}
```

Création :

```
let appareil = Appareil(
    nom: "iPhone",
    batterie: 85
)
```

4. Valeurs par défaut et init

Les deux approches peuvent être combinées.

```
class Appareil {
    var nom: String
    var batterie: Int
    var estConnecte: Bool = false

    init(nom: String, batterie: Int) {
        self.nom = nom
        self.batterie = batterie
    }
}
```

Création :

```
let appareil = Appareil(
    nom: "iPhone",
    batterie: 80
)
```

La propriété estConnecte reçoit automatiquement false.

5. Plusieurs initialisateurs

Une classe peut avoir plusieurs initialisateurs.

Exemple :

```
class Utilisateur {
    var nom: String
    var age: Int

    init(nom: String, age: Int) {
        self.nom = nom
        self.age = age
    }

    init(nom: String) {
        self.nom = nom
        self.age = 0
    }
}
```

On peut maintenant créer :

```
let utilisateur1 = Utilisateur(
    nom: "Alex",
    age: 25
)
```

ou :

```
let utilisateur2 = Utilisateur(nom: "Marie")
```

6. Surcharger un initialisateur

Les initialisateurs peuvent avoir le même nom :

```
init
```

mais des paramètres différents.

Exemple :

```
init(nom: String)
```

et :

```
init(nom: String, age: Int)
```

Swift choisit l'initialisateur selon les arguments fournis.

7. Propriétés stockées

Une propriété stockée conserve directement une valeur.

Exemple :

```
class Appareil {
    var batterie: Int = 100
}
```

La valeur 100 est réellement stockée dans l'objet.

8. Propriétés calculées

Une propriété calculée ne stocke pas directement une valeur. Elle la calcule lorsqu'elle est utilisée.

Exemple :

```
class Rectangle {
    var largeur: Double
    var hauteur: Double

    init(largeur: Double, hauteur: Double) {
        self.largeur = largeur
        self.hauteur = hauteur
    }

    var aire: Double {
        return largeur * hauteur
    }
}
```

9. Utiliser une propriété calculée

```
let rectangle = Rectangle(
    largeur: 10,
    hauteur: 5
)

print(rectangle.aire)
```

Résultat :

```
Console
50.0
```

On utilise rectangle.aire et non rectangle.aire(). Pourquoi ? Parce que aire est une propriété, pas une méthode.

10. Propriété stockée vs propriété calculée

Propriété stockée

```
var largeur: Double
```

La valeur est stockée.

Propriété calculée

```
var aire: Double {
    return largeur * hauteur
}
```

La valeur est calculée à partir d'autres données.

11. Exemple — Pourcentage de stockage utilisé

```
class Stockage {
    var capaciteTotale: Double
    var espaceDisponible: Double

    init(
        capaciteTotale: Double,
        espaceDisponible: Double
    ) {
        self.capaciteTotale = capaciteTotale
        self.espaceDisponible = espaceDisponible
    }

    var espaceUtilise: Double {
        return capaciteTotale - espaceDisponible
    }
}
```

Utilisation :

```
let stockage = Stockage(
    capaciteTotale: 128,
    espaceDisponible: 48
)

print(stockage.espaceUtilise)
```

Résultat :

```
Console
80.0
```

12. Propriété calculée avec plusieurs opérations

```
var pourcentageUtilise: Double {
    let utilise =
        capaciteTotale - espaceDisponible

    return (utilise / capaciteTotale) * 100
}
```

Utilisation :

```
print(
    "Stockage utilisé : \(stockage.pourcentageUtilise)%"
)
```

13. get

Une propriété calculée peut utiliser explicitement :

```
get
```

Exemple :

```
var aire: Double {
    get {
        return largeur * hauteur
    }
}
```

C'est équivalent à :

```
var aire: Double {
    return largeur * hauteur
}
```

Lorsque seule une lecture est nécessaire, get peut être omis.

14. Propriété calculée avec get et set

Une propriété calculée peut également accepter une nouvelle valeur.

Exemple :

```
class Temperature {
    var celsius: Double = 0

    var fahrenheit: Double {
        get {
            return (celsius * 9 / 5) + 32
        }
```

```
        set {
            celsius = (newValue - 32) * 5 / 9
        }
    }
}
```

15. Utiliser get

```
let temperature = Temperature()
temperature.celsius = 20
print(temperature.fahrenheit)
```

Résultat :

```
Console
68.0
```

Swift utilise automatiquement get lorsqu'on lit temperature.fahrenheit.

16. Utiliser set

```
temperature.fahrenheit = 86
```

Swift exécute set. Puis :

```
print(temperature.celsius)
```

donne environ :

```
Console
30.0
```

17. newValue

Dans :

```
set {
    celsius = (newValue - 32) * 5 / 9
}
```

newValue représente la nouvelle valeur affectée à la propriété.

Avec :

```
temperature.fahrenheit = 86
```

Swift considère :

```
newValue = 86
```

18. Nommer la valeur du set

On peut remplacer newValue par un nom explicite.

```
set(nouvelleTemperature) {
    celsius =
        (nouvelleTemperature - 32) * 5 / 9
}
```

Cela peut améliorer la lisibilité.

19. Propriétés en lecture seule

Une propriété calculée sans set est en lecture seule.

Exemple :

```
var niveauCritique: Bool {
    return batterie <= 10
}
```

On peut écrire :

```
if appareil.niveauCritique {
    print("Batterie critique")
}
```

Mais on ne peut pas écrire :

```
appareil.niveauCritique = true
```

car cette valeur est calculée.

20. Exemple — État de la batterie

```
class Batterie {
    var niveau: Int

    init(niveau: Int) {
        self.niveau = niveau
    }

    var estFaible: Bool {
        return niveau <= 20
    }

    var description: String {
        if niveau <= 10 {
            return "Critique"
        } else if niveau <= 20 {
            return "Faible"
```

```
        } else {
            return "Normale"
        }
    }
}
```

Utilisation :

```
let batterie = Batterie(niveau: 15)
print(batterie.estFaible)
print(batterie.description)
```

Résultat :

```
Console
true
Faible
```

Ces informations sont calculées à partir de niveau.

21. Propriétés de type avec static

Jusqu'ici, les propriétés appartiennent à chaque objet.

Exemple :

```
class Appareil {
    var batterie: Int = 100
}
```

Chaque instance possède sa propre batterie.

22. Propriété appartenant à la classe

Avec static, la propriété appartient au type lui-même.

Exemple :

```
class Appareil {
    static var nombreAppareils = 0
    var nom: String

    init(nom: String) {
        self.nom = nom
        Appareil.nombreAppareils += 1
    }
}
```

23. Accéder à une propriété static

On n'utilise pas :

```
iphone.nombreAppareils
```

On utilise :

```
Appareil.nombreAppareils
```

Exemple :

```
let iphone = Appareil(nom: "iPhone")
let ipad = Appareil(nom: "iPad")
print(Appareil.nombreAppareils)
```

Résultat :

```
Console
2
```

24. Instance vs type

Propriété d'instance

```
var nom: String
```

Chaque objet possède sa propre valeur :

```
iphone.nom
ipad.nom
```

Propriété de type

```
static var nombreAppareils = 0
```

Une seule valeur est associée au type :

```
Appareil.nombreAppareils
```

25. Méthode static

Une méthode peut également appartenir au type.

```
class Validation {
    static func niveauValide(_ niveau: Int) -> Bool {
        return niveau >= 0 && niveau <= 100
    }
}
```

Utilisation :

```swift
let valide = Validation.niveauValide(75)
```

Aucun objet Validation n'est nécessaire.

26. Exemple — Validation

```swift
class Validation {
    static func batterieValide(
        _ batterie: Int
    ) -> Bool {
        return batterie >= 0 &&
            batterie <= 100
    }

    static func stockageValide(
        _ stockage: Double
    ) -> Bool {
        return stockage >= 0
    }
}
```

Utilisation :

```swift
if Validation.batterieValide(85) {
    print("Valeur valide")
}
```

27. Contrôle d'accès

L'encapsulation ne consiste pas seulement à placer les données dans une classe. Il faut également contrôler qui peut accéder à quoi.

Swift propose plusieurs niveaux d'accès. Les principaux que nous utiliserons sont :

```swift
private
internal
public
```

28. internal

Par défaut, les éléments Swift sont généralement :

```swift
internal
```

Exemple :

```swift
var nom: String
```

sans modificateur explicite. Cela signifie essentiellement que l'élément est accessible dans le module courant.

Dans nos projets simples, il n'est généralement pas nécessaire d'écrire explicitement internal.

29. private

private limite fortement l'accès.

Exemple :

```
class Compte {
    private var solde: Double = 0
}
```

Ceci n'est alors pas accessible directement depuis l'extérieur :

```
let compte = Compte()
print(compte.solde)
```

Le code provoque une erreur d'accès.

30. Pourquoi utiliser private ?

Supposons :

```
class Batterie {
    var niveau: Int = 100
}
```

N'importe quel code peut faire :

```
batterie.niveau = -500
```

Ce qui crée un état invalide.

Avec :

```
private var niveau: Int = 100
```

on peut obliger l'utilisation de méthodes contrôlées.

31. Encapsulation plus forte

```
class Batterie {
    private var niveau: Int = 100

    func consommer(_ quantite: Int) {
        if quantite > 0 {
            niveau -= quantite
        }
        if niveau < 0 {
            niveau = 0
        }
    }
```

```
    func obtenirNiveau() -> Int {
        return niveau
    }
}
```

L'extérieur ne peut plus modifier directement niveau. Il doit passer par les comportements fournis.

32. private(set)

Swift permet une variante très pratique :

```
private(set)
```

Exemple :

```
class Batterie {
    private(set) var niveau: Int = 100
}
```

Action	Autorisation
Lecture depuis l'extérieur	Autorisée
Modification depuis l'extérieur	Interdite

33. Exemple avec private(set)

```
class Batterie {
    private(set) var niveau: Int

    init(niveau: Int) {
        self.niveau = niveau
    }

    func consommer(_ quantite: Int) {
        niveau -= quantite
        if niveau < 0 {
            niveau = 0
        }
    }
}
```

Utilisation :

```
let batterie = Batterie(niveau: 100)
print(batterie.niveau)
```

est permis. Mais :

```
batterie.niveau = 500
```

ne l'est pas.

34. Pourquoi private(set) est intéressant ?

Cela permet de créer une propriété visible mais contrôlée.

Exemples mobiles :

- Niveau batterie
- Progression téléchargement
- Solde
- Nombre de notifications
- État d'une connexion

Le reste de l'application peut lire l'état, mais seules certaines méthodes peuvent le modifier.

35. Exemple — Téléchargement

```
class Telechargement {
    private(set) var progression: Int = 0

    var estTermine: Bool {
        return progression >= 100
    }

    func avancer(de valeur: Int) {
        progression += valeur
        if progression > 100 {
            progression = 100
        }
    }
}
```

Utilisation :

```
let telechargement = Telechargement()
telechargement.avancer(de: 25)
print(telechargement.progression)
```

Résultat :

```
Console
25
```

Puis :

```
telechargement.avancer(de: 75)
print(telechargement.estTermine)
```

Résultat :

```
Console
true
```

36. Propriétés avec observateurs

Swift permet également d'exécuter du code lorsqu'une propriété change.

Deux observateurs existent :

```
willSet
didSet
```

37. willSet

willSet est exécuté avant la modification.

```
var batterie: Int = 100 {
    willSet {
        print("Nouvelle valeur : \(newValue)")
    }
}
```

38. didSet

didSet est exécuté après la modification.

```
var batterie: Int = 100 {
    didSet {
        print("Ancienne valeur : \(oldValue)")
    }
}
```

Les deux observateurs peuvent être combinés sur la même propriété :

```
var volume: Int = 50 {
    willSet {
        print("Le volume passera à \(newValue)")
    }
    didSet {
        print("Ancien volume : \(oldValue)")
    }
}
```

39. oldValue et newValue

Avec :

```
batterie = 80
```

Bloc	Valeur disponible
willSet	newValue → 80
didSet	oldValue → 100

40. Exemple — Surveillance de batterie

```
class Appareil {
    var batterie: Int = 100 {
        didSet {
            if batterie <= 20 {
                print("Attention : batterie faible")
            }
        }
    }
}
```

Utilisation :

```
let appareil = Appareil()
appareil.batterie = 50
appareil.batterie = 15
```

Lors du passage à 15 :

```
Console
Attention : batterie faible
```

41. Exemple complet — Appareil mobile mieux encapsulé

```
class AppareilMobile {
    static var nombreAppareils: Int = 0
    let nom: String
    private(set) var batterie: Int
    private(set) var stockageDisponible: Double

    var batterieFaible: Bool {
        return batterie <= 20
    }

    var stockageFaible: Bool {
```

```
        return stockageDisponible < 10
    }

    init(
        nom: String,
        batterie: Int,
        stockageDisponible: Double
    ) {
        self.nom = nom
        self.batterie = batterie
        self.stockageDisponible =
            stockageDisponible
        AppareilMobile.nombreAppareils += 1
    }

    func utiliserBatterie(_ quantite: Int) {
        if quantite > 0 {
            batterie -= quantite
        }
        if batterie < 0 {
            batterie = 0
        }
    }

    func installerApplication(
        taille: Double
    ) -> Bool {
        if taille > 0 &&
            taille <= stockageDisponible {
            stockageDisponible -= taille
            return true
        }
        return false
    }
}
```

42. Utilisation de la classe

```
let iphone = AppareilMobile(
    nom: "iPhone",
    batterie: 80,
    stockageDisponible: 50
)
```

Lecture autorisée :

```
print(iphone.batterie)
print(iphone.stockageDisponible)
```

Modification directe interdite :

```
iphone.batterie = 100
```

Modification contrôlée :

```
iphone.utiliserBatterie(20)
```

43. Propriétés calculées dans l'exemple

```
if iphone.batterieFaible {
    print("Batterie faible")
}
```

et :

```
if iphone.stockageFaible {
    print("Stockage faible")
}
```

Ces valeurs ne sont pas stockées séparément. Elles sont calculées à partir de l'état courant.

44. Propriété de type

Après plusieurs créations :

```
let iphone = AppareilMobile(...)
let ipad = AppareilMobile(...)
let montre = AppareilMobile(...)
```

on peut consulter :

```
print(
    AppareilMobile.nombreAppareils
)
```

Résultat :

```
Console
3
```

45. Exemple guidé — Gestion d'une batterie

Créer :

```
class Batterie
```

avec :

```
private(set) var niveau: Int
```

Ajouter :

```
init(niveau: Int)
```

Ajouter une propriété calculée :

```
var etat: String
```

Règles :

Niveau	État
0 à 10	Critique
11 à 20	Faible
21 à 100	Normale

Ajouter :

```
func consommer(_ quantite: Int)
```

La valeur doit toujours rester entre 0 et 100.

Créer également :

```
static func niveauValide(_ niveau: Int) -> Bool
```

46. Exercice — Gestion d'un téléchargement

Créer une classe :

```
class Telechargement
```

avec les propriétés :

```
nomFichier
tailleTotale
progression
```

1. Encapsulation

nomFichier doit être une constante :

```
let
```

La progression doit être :

```
private(set)
```

Elle commence à 0.

2. Initialisateur

Créer :

```
init(
    nomFichier: String,
    tailleTotale: Double
)
```

3. Propriété calculée

Créer :

```
var estTermine: Bool
```

qui retourne true lorsque progression >= 100.

Créer également :

```
var etat: String
```

Règles :

Progression	État
`0`	En attente
`1 à 99`	En cours
`100`	Terminé

4. Méthode

Créer :

```
func avancer(de valeur: Int)
```

La progression doit augmenter. Elle ne doit jamais dépasser 100 et ne doit jamais devenir négative.

5. Propriété static

Ajouter :

```
static var nombreTelechargements: Int
```

Cette propriété doit compter le nombre d'objets créés.

6. Affichage attendu

Exemple :

```
Console
------ TÉLÉCHARGEMENT ------
Fichier : update.zip
Taille : 250.0 Mo
Progression : 75 %
```

```
État : En cours
Nombre de téléchargements : 2
```

47. Exercice — Observateur de progression

Ajouter un :

```
didSet
```

sur la progression.

Lorsque la progression atteint ou dépasse 50 % pour la première fois, afficher :

Console
```
La moitié du téléchargement est atteinte.
```

Lorsque progression == 100, afficher :

Console
```
Téléchargement terminé.
```

48. À retenir

Initialisation

```
init(...) {
}
```

permet de construire un objet dans un état valide. Une classe peut disposer de plusieurs initialisateurs.

Propriété stockée

```
var batterie: Int
```

conserve une valeur.

Propriété calculée

```
var estFaible: Bool {
    return batterie <= 20
}
```

produit une valeur à partir d'autres propriétés.

get / set

```
var valeur: Double {
    get {
        // lecture
    }
    set {
        // modification
    }
}
```

Propriété de type

```
static var compteur = 0
```

appartient au type et non à une instance particulière.

Contrôle d'accès

Mot-clé	Effet
`private`	Cache une propriété ou une méthode
`private(set)`	Autorise la lecture depuis l'extérieur mais protège la modification

Observateurs

willSet et didSet permettent de réagir à la modification d'une propriété.

49. Vision globale

Nous avons maintenant enrichi les classes Swift :

```
Classe
  ↓
Objet
  ↓
Propriétés et méthodes
  ↓
Initialisation
  ↓
Propriétés stockées
  ↓
Propriétés calculées
  ↓
get / set
  ↓
static
  ↓
Encapsulation
  ↓
willSet / didSet
```

La prochaine étape logique est maintenant une notion particulièrement importante en Swift :

À venir : struct vs class.

Nous verrons notamment :

- Structures
- Types valeur
- Types référence
- Copie
- Partage de références
- mutating
- Identité des objets

C'est une distinction fondamentale du langage Swift avant d'aborder ensuite l'héritage et le polymorphisme.